# CS110 Week 11 — Unity Game Development Lecture Notes

## 1. Introduction to Game Development

- Game development is one of the most complex forms of programming.
- Game development combines:
    - Physics and mathematics
    - Human behavior and artificial intelligence
    - User interfaces (UI)
    - Object-oriented programming
    - Graphics and animation
    - Computer hardware and software
- Learning game development can help programmers understand other areas of software development.
- The instructor emphasized thinking like a creator or software architect, not just a programmer.

## 2. Variables, Functions, Classes, and Objects

- Variables: Containers that hold information.
- Functions/Methods: Actions that perform tasks.
- Classes: Blueprints that define how objects are created and behave.
- Objects: Instances created from classes.
- Properties: Control access to variables and other data.
- Classes contain variables, methods, and programming logic.
- Object-oriented programming organizes code around classes and objects.
- In game development, the instructor recommends thinking about classes before individual variables and functions.

## 3. Understanding the Technology Stack

- All programming ultimately depends on computer hardware and binary operations.
- Different programming languages use similar fundamental concepts.
- New technologies build on existing technologies.
- Programmers should connect new knowledge to concepts they already understand.
- Understanding the underlying system makes learning new programming languages easier.
- The instructor emphasized developing a mental map of how technologies connect.

# 4. Unity and Visual Studio

- Unity: Game engine used to develop games.
- Unity Hub: Application used to install and manage Unity versions and projects.
- Visual Studio: IDE used to write and edit C# code.
- C#: Programming language used for Unity scripting.
- Unity integrates with Visual Studio.
- Knowledge of C# from previous assignments applies to Unity.
- Unity also supports version control systems such as Git.
- GitHub allows teams to store, share, and manage their code.

# 5. Unity Interface

- Scene View: Used to create and arrange objects.
- Game View: Shows what the player sees.
- Hierarchy: Lists the GameObjects in the current scene.
- Inspector: Displays and allows editing of an object's components and properties.
- Project Window: Contains scripts, assets, materials, and other project files.
- Console: Displays debugging information, warnings, and errors.
- Unity's interface has similarities to Visual Studio.
- Unity provides tools that simplify game development.

# 6. Game Engines and Game Loops

- Game engines provide systems needed to create games.
- Without a game engine, developers would need to build many systems themselves.
- A game loop continuously updates the game.
- Games constantly update movement, rotation, animations, and other behaviors.
- Unity provides a built-in game loop.
- Traditional applications often wait for user actions, while games usually update continuously.
- Unity is designed to manage real-time game behavior.

# 7. Start() and Update()

- Start():
    - Called once before the first frame update when the component is enabled.
    - Used to initialize variables and starting conditions.
    - Can set an object's starting color.
- Update():
    - Called once every frame while the component is active and enabled.
    - Used for repeated actions.

    - Can control movement and rotation.
- Unity automatically calls these methods.
- `Time.deltaTime` helps make movement independent of frame rate.

## 8. GameObjects and Components

- Everything placed in a Unity scene is generally represented by a GameObject.
- GameObjects can represent players, enemies, cameras, lights, or other objects.
- Components add functionality to GameObjects.
- A GameObject can contain multiple components.
- C# scripts can be attached to GameObjects as components.
- Components allow developers to organize different behaviors separately.
- Unity uses object-oriented programming concepts.

## 9. Transform, Movement, and Rotation

- Every GameObject has a Transform component.
- Transform controls:
    - Position: Where the object is located.
    - Rotation: The direction the object faces.
    - Scale: The size of the object.
- `transform.Translate()` moves an object.
- `transform.Rotate()` rotates an object.
- Movement speed can be controlled using variables.
- Rotation speed can be measured in degrees per second.
- Developers can adjust exposed values through the Inspector.

## 10. Public, Private, and Properties

- Public: Accessible from outside a class, subject to C# access rules.
- Private: Accessible only within the declaring class.
- Properties provide controlled access to data.
- Encapsulation helps protect variables from unintended changes.
- Public fields can be displayed in Unity's Inspector.
- Private fields can also be displayed using `[SerializeField]`.
- The instructor emphasized understanding how classes protect and organize data.

## 11. Understanding `this` and `transform`

- `this` is a C# keyword referring to the current object instance.

- `transform` is a Unity property that provides access to the object's Transform component.
- `this.transform` explicitly accesses that property on the current instance.
- `transform.Translate()` moves the current GameObject.
- `transform.Rotate()` rotates the current GameObject.
- The instructor emphasized understanding naming conventions and object references.
- Lowercase names do not necessarily mean that a member is private.

## 12. Changing Object Colors

- Unity provides built-in systems for controlling materials and colors.
- A Renderer component controls how an object is displayed.
- A material determines aspects of an object's appearance.
- `material.color` can be used to change a material's color when supported by its shader.
- Color changes can be programmed using C#.
- The instructor discussed three main behaviors:
    - Movement
    - Rotation
    - Color changes

## 13. Prefabs

- Prefabs: Reusable templates for GameObjects.
- Prefabs allow developers to create multiple objects from the same template.
- Each prefab instance can have different properties.
- Examples include different positions, rotations, colors, and speeds.
- Prefabs help organize and reuse objects.
- Developers can update a prefab and apply changes to its instances.
- Prefabs simplify managing large numbers of similar objects.
- Each instantiated prefab still exists as a separate GameObject and uses runtime resources.

## 14. Unity Versus Unreal Engine

- Unity primarily uses C# for gameplay scripting.
- Unreal Engine primarily uses C++ and Blueprints.
- The instructor described Unity as easier for beginners to approach.
- He described Unreal as powerful for developing AAA games.
- Both engines support object-oriented programming.
- Both can create complex games.

- The instructor emphasized that engine architecture affects usability and development workflows.

## 15. Unity UI and WPF

- The instructor connected Unity's UI development to earlier lessons about WPF, HTML, and CSS.
- WPF is used for creating Windows desktop interfaces.
- HTML and CSS are used for creating web interfaces.
- Unity has its own UI systems.
- Unity's UI Toolkit uses concepts similar to web development, including styling and layout.
- Existing programming knowledge can help developers learn new interface systems.

## 16. Teamwork and GitHub

- Game development often requires teamwork.
- Teams need consistent project organization.
- Git tracks changes to project files.
- GitHub allows team members to share their work.
- Teams should establish clear development procedures.
- Consistent folder structures help prevent confusion.
- The instructor described creating a C# utility that automatically generated folders for his development team.
- Small custom utilities can improve team productivity.

## 17. Artificial Intelligence in Programming

- AI can help programmers develop applications and utilities.
- Programmers still need to understand how systems work.
- AI-generated code should be reviewed and tested.
- Teams can use AI to automate repetitive tasks.
- AI can help create custom tools for specific projects.
- The instructor emphasized that understanding programming makes AI more useful.
- Developers should use AI as a tool rather than relying on it without understanding the results.

## 18. The Instructor's Programming Philosophy

- Programming involves more than memorizing syntax.
- Students should understand how technologies relate to one another.
- Learning new technology becomes easier when connected to existing knowledge.
- Programmers should visualize how systems work.

- Software development often involves solving complicated problems.
- Understanding underlying concepts helps programmers adapt to new tools.
- Teams should create organized workflows and procedures.
- The instructor emphasized becoming a creator and problem-solver rather than simply completing assignments.

# 19. Week 11 Unity Assignment

- Create a Unity project and scene.
- Create a cube named `MyMarker`.
- Write or use three C# scripts:
    - `TranslateObject.cs`
    - `RotateObject.cs`
    - `ChangeObjectColor.cs`
- Attach the scripts to the cube.
- Test movement, rotation, and color changes.
- Change values in the Inspector.
- Observe how different values affect the object.
- Create a prefab.
- Create two instances with different settings.
- Record observations and screenshots.
- Save the work in GitHub.
- Share findings with the team.